Safety, Security e Resiliência

Prof. Dr. Iaçanã Ianiski Weber

Confiabilidade e Segurança de Software

98G08-4

(Material com referências abertas; verificar licenças de figuras antes de incorporar.)

Índice

1. Motivação e mapa conceitual
2. Métricas e trade-offs de dependability
3. Safety engineering: de hazard a requisito e evidência
4. Security engineering: de threat model a controles e verificação
5. Resiliência (incl. cyber resiliency) como requisito arquitetural
6. Modelagem aplicada: PSA (Platform Security Architecture)
7. Fechamento e gancho para o próximo deck (Teste de Software)

Índice

1. Motivação e mapa conceitual
2. Métricas e trade-offs de dependability
3. Safety engineering: de hazard a requisito e evidência
4. Security engineering: de threat model a controles e verificação
5. Resiliência (incl. cyber resiliency) como requisito arquitetural
6. Modelagem aplicada: PSA (Platform Security Architecture)
7. Fechamento e gancho para o próximo deck (Teste de Software)

Motivação

- Sistemas modernos são **ciberfísicos** e **conectados**: falhas + ataques podem gerar **danos físicos**.
- Dependability (confiabilidade) não é "só bug": envolve **hardware, software, rede, humano, processo**.
- **Convergência**: safety (acidental) + security (malicioso) + resiliência (sobrevivência e recuperação).
- A indústria já formalizou isso: a **PSA** (*Platform Security Architecture*) da Arm define um framework de análise de ameaças, requisitos de segurança e implementação de referência para dispositivos conectados — mostrando que **security by design** é exigência de mercado, não opcional.

Objetivo da aula: fornecer um **framework** para identificar e justificar requisitos (safety/security) e decisões arquiteturais (resiliência).

Dependability/Trustworthiness: visão unificadora (taxonomia clássica)

Ideia-chave

Dependability é um conceito guarda-chuva que inclui atributos como: *reliability, availability, safety, integrity, maintainability* (e, ao considerar security, entra *confidentiality*).

- **Atributos (o que queremos)**: disponibilidade, confiabilidade, segurança funcional, integridade, etc.
- **Ameaças (o que pode dar errado)**: *fault* $\rightarrow$ *error* $\rightarrow$ *failure*; e ataques.
- **Meios (como alcançamos)**: prevenção, tolerância a falhas, remoção de falhas, previsão/estimativa.

Fonte: Avizienis et al., IEEE TDSC 2004.

Safety vs Security: definição e diferença operacional

Safety (acidental)

- Foco: **evitar dano** a pessoas/ambiente.
- Causa típica: **falha não intencional** (bug, desgaste, erro humano).
- Pergunta: "*o que pode causar um acidente?*" (hazard analysis)

Security (malicioso)

- Foco: **proteger ativos** contra adversários.
- Causa típica: **ação intencional** (exploit, engenharia social, supply chain).
- Pergunta: "*como um atacante consegue isso?*" (threat modeling)

Interseção crítica: ataques podem gerar **hazards** — por exemplo, um atacante que compromete a rede CAN de um veículo pode injetar mensagens falsas de frenagem, transformando uma vulnerabilidade de *security* em risco direto à vida (*safety*). *Veja na prática:* Hacking Jeep

Ameaças: falhas vs ataques (vocabulário)

Mundo de falhas (dependability clássica)

- **Fault**: defeito/causa (HW/SW/humano).
- **Error**: estado incorreto interno.
- **Failure**: desvio observado do serviço esperado.

Mundo de ataques (security)

- **Threat**: adversário + capacidade + intenção.
- **Vulnerability**: fraqueza explorável.
- **Exploit**: método para violar uma propriedade.

Ponte entre os mundos

Ataque explora **vulnerabilidade** e induz **erro** (estado incorreto), resultando em **falha** (failure) que pode virar **acidente** (safety).

Índice

1. Motivação e mapa conceitual
2. Métricas e trade-offs de dependability
3. Safety engineering: de hazard a requisito e evidência
4. Security engineering: de threat model a controles e verificação
5. Resiliência (incl. cyber resiliency) como requisito arquitetural
6. Modelagem aplicada: PSA (Platform Security Architecture)
7. Fechamento e gancho para o próximo deck (Teste de Software)

Métricas clássicas: Reliability e Availability

Reliability (sobrevivência até t)

$$R(t) = P(T > t)$$

Para taxa de falhas constante λ (modelo exponencial):

$$R(t) = e^{-\lambda t}$$

Availability (tempo operacional)

$$A \approx \frac{MTBF}{MTBF + MTTR}$$

- MTBF: tempo médio entre falhas
- MTTR: tempo médio de reparo/recuperação

Integridade, Safety e Security: métricas e "o que medir"

- **Integrity**: correção e não corrupção (dados/estado/comando).
- **Safety**: probabilidade de **dano** + severidade (frequentemente via classes de risco).
- **Security**: não há uma única métrica; use **objetivos** (CIA/AAA), risco, cobertura de controles, e métricas operacionais (MTTD/MTTR).
  - **CIA**: *Confidentiality* (acesso só a quem autorizado), *Integrity* (dados não adulterados), *Availability* (sistema acessível quando necessário).
  - **AAA**: *Authentication* (verificar identidade), *Authorization* (controlar permissões), *Accountability* (rastrear ações).

Atenção

Métricas precisam ser **amarradas a um objetivo** e a um **modelo de ameaça/uso**. Sem isso viram "números bonitos".

Trade-offs típicos

- **Autenticação** pode aumentar **latência** (impacto em controle em tempo real).
- **Criptografia** aumenta custo/energia; pode afetar disponibilidade em HW fraco.
- **Redundância** melhora **availability**, mas pode piorar **reliability** se introduzir mais componentes/falhas.

Engenharia = justificar trade-offs

Não é "ter tudo", é **definir o que é aceitável** e **provar com evidência**.

Índice

1. Motivação e mapa conceitual
2. Métricas e trade-offs de dependability
3. Safety engineering: de hazard a requisito e evidência
4. Security engineering: de threat model a controles e verificação
5. Resiliência (incl. cyber resiliency) como requisito arquitetural
6. Modelagem aplicada: PSA (Platform Security Architecture)
7. Fechamento e gancho para o próximo deck (Teste de Software)

Safety: conceitos operacionais

- **Hazard**: condição/estado do sistema que, combinada com o ambiente, pode levar a dano.
- **Accident/Loss**: dano efetivo (evento de perda).
- **Safety constraint**: restrição que deve ser mantida para evitar hazards.

Safety não é "zero risco"

É **reduzir risco a um nível aceitável** (e justificar isso).

Ciclo de vida e padrões (o que a indústria exige)

- Safety-critical usa **ciclo de vida** + **gestão de configuração** + **rastreabilidade**.
- Padrões variam por domínio: industrial (*IEC 61508*), automotivo (*ISO 26262*), aeronáutico (*DO-178C/ED-12C*).

Artefatos típicos

Hazard log, safety requirements, arquitetura de mitigação, V&V plan, evidências, **safety case**.

Ferramentas de Hazard Analysis

FMEA — *Failure Mode and Effects Analysis*

- Abordagem **bottom-up**: parte dos componentes e pergunta "o que acontece se isso falhar?"
- Identifica modos de falha individuais e seus efeitos no sistema.
- Adequado para HW/SW com fronteiras bem definidas entre componentes.

FTA — *Fault Tree Analysis*

- Abordagem **top-down**: parte de um evento indesejado e busca as causas raízes.
- Representa combinações lógicas de falhas (portas AND/OR) que levam ao acidente.
- Útil para justificar redundâncias e demonstrar que combinações críticas são improváveis.

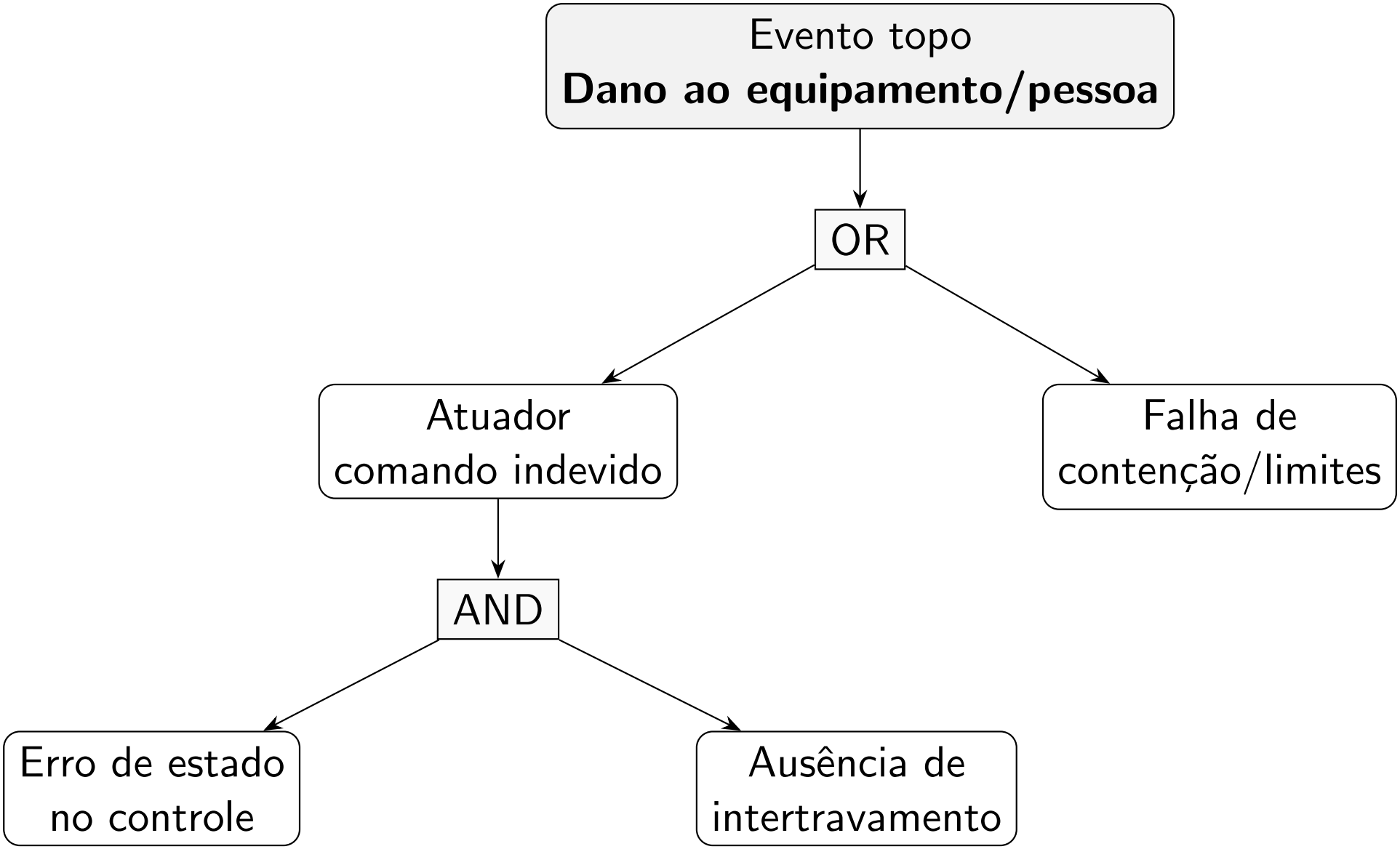
Exemplo de FMEA

Item	Modo de falha	Efeito	Mitigação
Sensor de temperatura	leitura travada (*stuck-at*)	superaquecimento não detectado	redundância de sensores + alarme independente
Válvula de resfriamento	falha em posição fechada	temperatura sobe sem controle	válvula normalmente aberta (*fail-safe*)
Controlador (PLC)	reinicialização inesperada	perda de controle transitória	watchdog + estado seguro na inicialização
Rede de campo	perda de mensagem	atuador recebe comando obsoleto	timeout + retorno ao modo local

Ponto crítico

Mitigação vira **requisito verificável** (e entra no plano de V&V).

Fault Tree Analysis (FTA): do acidente às causas



Índice

1. Motivação e mapa conceitual
2. Métricas e trade-offs de dependability
3. Safety engineering: de hazard a requisito e evidência
4. Security engineering: de threat model a controles e verificação
5. Resiliência (incl. cyber resiliency) como requisito arquitetural
6. Modelagem aplicada: PSA (Platform Security Architecture)
7. Fechamento e gancho para o próximo deck (Teste de Software)

Security: propriedades e fronteiras de confiança

- Objetivos clássicos: **CIA** (Confidentiality, Integrity, Availability) + **AAA** (Authentication, Authorization, Accounting).
- Em embarcados/ciberfísicos, **integridade de comando** e **tempo** (freshness) são críticos.
- Segurança começa com **trust boundaries**: onde dados/comandos cruzam domínios.

STRIDE: categorias de ameaças

Categoria	Descrição	Propriedade violada
Spoofing	falsificar identidade de usuário ou componente	Authentication
Tampering	modificar dados ou código sem autorização	Integrity
Repudiation	negar ter realizado uma ação	Accountability
Information disclosure	expor dados a quem não tem permissão	Confidentiality
Denial of Service	tornar o sistema indisponível	Availability
Elevation of privilege	obter permissões além do autorizado	Authorization

STRIDE é um ponto de partida para identificar ameaças — cada categoria orienta perguntas sobre o sistema.

Threat Modeling

1. **Modelar o sistema** — desenhar o diagrama de fluxo de dados (DFD) com componentes, fluxos e *fronteiras de confiança* (ex: onde dados cruzam de rede externa para sistema interno).
2. **Identificar ameaças** — percorrer cada componente e fluxo com STRIDE: "o que um atacante pode fazer aqui?"
3. **Avaliar e priorizar** — estimar impacto e probabilidade de cada ameaça; focar esforço onde o risco é maior.
4. **Definir mitigações** — para cada ameaça priorizada, mapear um controle concreto (ex: autenticação mútua, criptografia, validação de entrada).

Importante

Threat modeling é **iterativo**: deve ser revisado a cada mudança relevante de arquitetura.

Risk Assessment em security: processo (NIST SP 800-30)

- Defina: escopo, ativos, missão, tolerância a risco (*risk framing*).
- Identifique: **threat sources/events**, **vulnerabilities**, **predisposing conditions**.
- Estime: **likelihood** e **impact**.
- Determine: **risk** e priorize respostas.

Entrega típica

Registro de riscos + plano de tratamento (mitigar, transferir, aceitar, evitar) + monitoramento.

Referência sugerida: NIST SP 800-30 Rev.1.

Mecanismos típicos (embedded/IoT)

- **Secure boot** + cadeia de confiança (root of trust).
- **Atualização segura** (assinatura, anti-rollback, A/B, recovery).
- **Proteção de chaves** (HSM/secure element/TPM quando aplicável).
- **Hardening**: superfície mínima, privilégios mínimos, isolamento (MPU/MMU).
- **Observabilidade**: logs auditáveis, telemetria, detecção de anomalia.

Verificação em security: do requisito ao teste

Exemplo de requisito: *"toda atualização deve ser autenticada e íntegra."*

Como gerar evidência para esse requisito:

- **Revisão de design + threat model** — verificar se a arquitetura cobre a ameaça de atualização maliciosa.
- **Testes dinâmicos** — fuzzing do parser de pacotes, testes de downgrade e rollback forçado.
- **Análise estática e de composição**
  - **SAST**: busca vulnerabilidades no código-fonte sem executar o programa.
  - **DAST**: testa o sistema em execução simulando ataques reais.
  - **SBOM** + revisão de dependências: lista todos os componentes e verifica vulnerabilidades conhecidas (CVEs).
- **Pentest** orientado ao modelo de ameaça — validação por tentativa de exploração real.

Quando security vira safety (e vice-versa)

Security ⇒ Safety:

- Em 2015, pesquisadores assumiram controle remoto de um Jeep Cherokee via vulnerabilidade no sistema de infotainment — acelerando, freando e desligando o motor em rodovia (recall de 1,4 milhão de veículos).
- Em 2021, o ataque ransomware à Colonial Pipeline interrompeu o fornecimento de combustível no leste dos EUA, afetando hospitais e serviços de emergência.

Safety ⇒ Security:

- Portas de manutenção e modos de bypass criados para safety (ex.: acesso de emergência sem autenticação) tornam-se vetores de ataque se não forem protegidos.
- Um gateway automotivo que exige autenticação forte pode **bloquear comandos de frenagem de emergência** se as credenciais falharem — a mitigação de security cria um hazard de safety.

Mensagem: projetar separadamente é arriscado — o correto é **co-analisar** interações e trade-offs.

Índice

1. Motivação e mapa conceitual
2. Métricas e trade-offs de dependability
3. Safety engineering: de hazard a requisito e evidência
4. Security engineering: de threat model a controles e verificação
5. Resiliência (incl. cyber resiliency) como requisito arquitetural
6. Modelagem aplicada: PSA (Platform Security Architecture)
7. Fechamento e gancho para o próximo deck (Teste de Software)

Resiliência: definição operacional

Definição (cyber resiliency)

Capacidade de **antecipar**, **resistir**, **recuperar** e **adaptar** a condições adversas, estresses, ataques ou compromissos envolvendo recursos cibernéticos.

- Resiliência não substitui safety/security: ela **complementa** quando a prevenção falha.
- Entra como requisitos de: **degradação graciosa**, **fail-operational**, **reconfiguração**, **continuidade**.

Referência sugerida: NIST SP 800-160 v2r1 (Developing Cyber-Resilient Systems).

Padrões arquiteturais de resiliência

Antecipar/Resistir

- segmentação/zonas
- least privilege / isolamento
- diversidade (N-version, heterogeneidade)
- rate limiting, circuit breakers
- validação robusta (parsers)

Recuperar/Adaptar

- rollback (A/B), recovery mode
- checkpoints, reinicialização controlada
- degradação graciosa (safe state)
- reconfiguração/roteamento alternativo
- observabilidade + aprendizado pós-incidente

Resiliência como "engenharia ao longo do ciclo de vida"

- Resiliência exige processos: governança, gestão de riscos, resposta a incidentes, melhorias contínuas.
- Em organizações maduras, security/resiliência são tratadas como **práticas contínuas** (não "projeto fechado").

Sugestão de leitura: relatórios do CMU/SEI sobre práticas de segurança/resiliência ao longo do ciclo de vida.

Índice

1. Motivação e mapa conceitual
2. Métricas e trade-offs de dependability
3. Safety engineering: de hazard a requisito e evidência
4. Security engineering: de threat model a controles e verificação
5. Resiliência (incl. cyber resiliency) como requisito arquitetural
6. Modelagem aplicada: PSA (Platform Security Architecture)
7. Fechamento e gancho para o próximo deck (Teste de Software)

PSA: o que é e por que importa

- **PSA** (*Platform Security Architecture*) é um framework criado pela Arm (2017) para guiar a construção de dispositivos IoT seguros.
- Não é só especificação de hardware: cobre **análise de ameaças**, **arquitetura**, **implementação** e **certificação**.
- Agnóstico de implementação — pode ser aplicado a qualquer chip, software ou dispositivo.

As 4 etapas do PSA

Analyze (modelar ameaças e requisitos) → **Architect** (projetar a segurança) → **Implement** (construir com referência aberta) → **Certify** (validar contra os 10 security goals).

PSA: os 10 Security Goals

1. **Unique Device Identity** — cada dispositivo com identidade única e atestável.
2. **Security Lifecycle** — estados de segurança rastreáveis (fabricação, operação, fim de vida).
3. **Attestation** — evidência verificável das propriedades do dispositivo.
4. **Secure Boot** — apenas código autorizado executa no boot.
5. **Secure Update** — atualizações autênticas e íntegras via OTA (*Over-The-Air*).
6. **Anti-Rollback** — impedir reinstalação de versões antigas vulneráveis.
7. **Isolation** — serviços confiáveis isolados entre si e do código não confiável.
8. **Interaction & Communication** — comunicação autenticada e cifrada.
9. **Secure Storage** — proteção de chaves, credenciais e dados sensíveis em repouso.
10. **Crypto Services** — primitivas criptográficas fornecidas pelo RoT (*Root of Trust*).

PSA: RoT e ambientes de processamento

- **RoT** (*Root of Trust*): componente de hardware/software que ancora as funções de segurança — identidade, criptografia, attestation e secure boot.
- O **PRoT** (*Platform Root of Trust*) é responsável por:
  - configurar a segurança no nível do sistema,
  - ancorar o secure boot (cadeia de confiança),
  - estabelecer o **SPE** (*Secure Processing Environment*).
- Modelo mínimo: dois ambientes isolados:
  - **SPE**: hospeda partições seguras (crypto, attestation, secure storage).
  - **NSPE** (*Non-Secure Processing Environment*): aplicação "normal".

PSA na prática: Analyze + Architect

Analyze — usar threat model + STRIDE para identificar ameaças ao dispositivo:

- Quais são os **assets**? (chaves, firmware, dados de sensores, identidade)
- Quais são as **trust boundaries**? (SPE/NSPE, rede, interface de debug, OTA)
- Quais ameaças STRIDE se aplicam a cada fronteira?

Architect — mapear ameaças aos 10 security goals:

- Firmware malicioso via OTA? → **Secure Update** + **Anti-Rollback** + **Secure Boot**.
- Extração de chaves? → **Secure Storage** + **Isolation** + **Crypto Services**.
- Spoofing de dispositivo? → **Unique Identity** + **Attestation**.

PSA na prática: Implement + Certify

Implement — referência aberta:

- **TF-M** (*Trusted Firmware-M*): implementação open-source do PSA para Cortex-M.
- Fornece serviços prontos: crypto, attestation, secure storage, isolamento via partições.
- Portável para diferentes chips e plataformas.

Certify — níveis de certificação PSA Certified:

- **Nível 1**: questionário de boas práticas (baseline).
- **Nível 2**: avaliação laboratorial do RoT contra ataques de software.
- **Nível 3**: resistência a ataques de hardware (side-channel, fault injection).